HDOC-58

Kth Smallest Element Using Pointers and Bubble Sort

Question / Problem Statement

Write a C program that reads the size and elements of a variable-length integer array arr[] and an integer k such that 0 < k <= size. Pass the array to a function named kth_smallest() using its address. Use pointers to pass the array between the calling function and kth_smallest(). The function must:

- Arrange the elements of arr[] in ascending order using bubble sort.
- Find the element that occupies the kth position in the sorted array.
- Display the kth smallest element.
- Consider duplicate elements as separate elements.
- Display -1 if k is invalid.

Algorithm / Logic

1. Read the size of the array.

2. Declare a variable-length integer array of the entered size and read its elements.

3. Read the value of k.

4. Pass the address of the array, size, and k to kth_smallest().

5. Inside kth_smallest(), first check whether k <= 0 or k > size. If invalid, return -1.

6. Use pointer arithmetic and bubble sort to arrange the array in ascending order.

7. After sorting, the kth smallest element is at index k - 1 because C arrays use zero-based indexing.

8. Return *(arr + k - 1). Duplicate values remain separate array elements.

9. Display the returned result in main().

Complete C Program

```
#include <stdio.h>

int kth_smallest(int *arr, int size, int k)
{
    int i, j, temp;

    if (k <= 0 || k > size)
        return -1;

    for (i = 0; i < size - 1; i++)
    {
        for (j = 0; j < size - i - 1; j++)
        {
            if (*(arr + j) > *(arr + j + 1))
            {
                temp = *(arr + j);
                *(arr + j) = *(arr + j + 1);
                *(arr + j + 1) = temp;
            }
        }
    }

    return *(arr + k - 1);
}

int main()
```

```
{
    int size, k, i, result;

    printf("Enter size of array: ");
    scanf("%d", &size);

    if (size <= 0)
    {
        printf("Invalid array size.\n");
        return 0;
    }

    int arr[size];

    printf("Enter %d elements: ", size);
    for (i = 0; i < size; i++)
        scanf("%d", &arr[i]);

    printf("Enter k: ");
    scanf("%d", &k);

    result = kth_smallest(arr, size, k);

    if (k <= 0 || k > size)
        printf("-1\n");
    else
    {
        printf("Sorted array: ");
        for (i = 0; i < size; i++)
            printf("%d ", arr[i]);
        printf("\nKth smallest element = %d\n", result);
    }

    return 0;
}
```

Explanation

The function kth_smallest() receives the base address of the array through the pointer arr. Therefore, the same array created in main() is directly accessed and modified inside the function.

Bubble sort repeatedly compares adjacent elements. The expressions *(arr + j) and *(arr + j + 1) access adjacent elements through pointer arithmetic. If the left element is greater, the two values are swapped.

After sorting in ascending order, the first element is the 1st smallest, the second is the 2nd smallest, and so on. Hence the kth smallest value is stored at *(arr + k - 1). Duplicates are not removed, so equal values occupy separate positions.

If k is outside the valid range 1 to size, the function returns -1 as required.

Test Cases

Test	Input Array	k	Sorted Array	Expected Result
1	7, 2, 9, 1, 5	3	1, 2, 5, 7, 9	5
2	4, 2, 2, 8, 1	3	1, 2, 2, 4, 8	2
3	10, 20, 30	1	10, 20, 30	10
4	5, 3, 7	4	-	-1
5	6, 6, 6, 6	2	6, 6, 6, 6	6

Sample Output

```
Enter size of array: 5
Enter 5 elements: 7 2 9 1 5
Enter k: 3
Sorted array: 1 2 5 7 9
Kth smallest element = 5
```

Result

The program successfully sorts the integer array in ascending order using bubble sort through pointers and finds the kth smallest element. Duplicate elements are treated as separate elements, and -1 is displayed when k is invalid.